

1.1 Что такое Java?

Java - это язык программирования, на котором пишут программы для серверов, сайтов, банковских систем, Android-приложений и внутренних корпоративных сервисов.

Ее главный плюс - надежность, кроссплатформенность, понятная структура и способность хорошо работать в больших проектах.

Java существует с 1995 года и до сих пор остается в топе популярных языков.

Где используется Java:

Альфа-Банк, Сбер, Т-Банк, Яндекс, Ozon, МТС. Занимает более 40% рынка

1.2 Что такое программа простыми словами?

Программа - это набор команд для компьютера.

Человек пишет: что взять, что посчитать, что проверить и что показать на экране.

Компьютер просто выполняет эти команды по правилам.

Пример:

калькулятор - программа;

телеграм - программа;

банковское приложение - тоже программа.

Разница только в масштабе и сложности.

1.3 Как начать писать код на Java?

Для старта нужны две вещи:

JDK - набор инструментов Java. Он нужен, чтобы писать, компилировать и запускать программы.

IDE - среда разработки. Это программа, в которой удобно писать код.

Самый популярный вариант для учебы и работы - **IntelliJ IDEA**.

Скачать ее можно тут - <https://www.jetbrains.com/idea/> (с RU IP – нельзя)

Активировать премиум версию можно так – <https://blog.linh9ra.ru/активация-phpstorm-webstorm-intellij-idea-и-другие-продукты-jetbrains-в/>

1.4 Как запустить самую простую программу на Java?

Пример самой простой программы:

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Привет, Java!");  
    }  
}
```

Что она делает? Просто выводит текст в консоль.

Через IDE: Создать проект → Создать класс Main → Вставить код → Нажать Run.

Через терминал:

```
javac Main.java  
java Main
```

Сначала код компилируется, потом запускается.

1.5 С чего начинается выполнение Java-программы? Метод main

Java-программа начинается не с первой строки файла, а с **метода main**. Именно его JVM ищет при запуске.

То есть логика такая:

запуск программы → JVM находит main → код внутри main выполняется

Это и есть стартовая точка обычного Java-приложения.

main - это главный стартовый метод программы.

Без него Java не поймет, откуда начинать выполнение.

Вот как он выглядит: **public static void main(String[] args) {}**

Разбор по частям: **public** - метод доступен для запуска, **static** - можно вызвать без создания объекта, **void** - ничего не возвращает, **String[] args** - сюда можно передать параметры при запуске JVM.

На старте достаточно запомнить главное - **main** - это вход в программу.



2.1 Что такое класс в Java?

Класс - это **чертеж**. Например, чертеж машины: на нем еще нет настоящей машины, но уже понятно, какие у нее есть части и что она умеет делать. В Java класс работает так же: он задает основу, внутри которой описываются данные и действия.

```
public class Car { no usages
    String color; no usages
    int speed; no usages

    void drive() { no usages
        System.out.println("Машина едет");
    }
}
```

Здесь Car - это класс. Мы как будто описали машину на бумаге: у нее есть цвет, скорость и действие drive().

Главная мысль простая: **класс - это шаблон, по которому строится код.**

2.2 Что такое метод?

Метод - это **действие внутри класса**.

Если класс - это чертеж машины, то метод - это то, что машина умеет делать: ехать, тормозить, сигналить.

```
public class Car { no usages
    void drive() { no usages
        System.out.println("Машина едет");
    }

    void stop() { no usages
        System.out.println("Машина остановилась");
    }
}
```

Здесь drive() и stop() - методы. Один отвечает за движение, другой - за остановку.

То есть класс отвечает на вопрос **“что это такое?”**, а метод - **“что это умеет делать?”**.

2.3 Что такое тело метода?

Тело метода - это код, который находится **внутри метода**, между фигурными скобками. Именно там записано, что должно произойти.

```
void drive() { no usages
    System.out.println("Машина завелась");
    System.out.println("Машина едет");
}
```

`void drive()` - это заголовок метода.

А две строки внутри `{}` - это его тело.

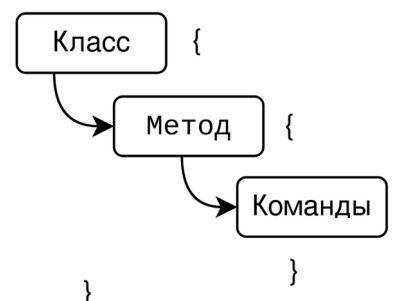
Если совсем просто: название метода говорит, **что делаем**, а тело метода показывает, **как это делаем**.

2.4 Зачем в Java нужны фигурные скобки {}?

Фигурные скобки нужны, чтобы показать границы кода.

Они помогают Java понять, где начинается класс, где начинается метод и где этот кусок кода заканчивается.

```
public class Car { no usages
    void drive() { no usages
        System.out.println("Машина едет");
    }
}
```



Первые скобки относятся к классу `Car`, вторые - к методу `drive()`.

2.5 Что делает `System.out.println()`?

`System.out.println()` - выводит текст или значение в консоль.

`System.out.println("Машина едет");` → **Результат** – вывод на экран Машина едет

Это один из первых способов “увидеть”, что программа реально работает.

Но здесь важно сразу подметить одну вещь на будущее:

- `println()` - это **метод**, который вызывается у `out`
- `out` - это специальный объект для вывода.
- А `System` - это класс, в котором этот объект хранится.

в Java такие записи - это не магия, а вызов метода у объекта.



2.6 Что такое комментарии и какие они бывают?

Комментарии - это текст в коде для человека.

Программа их не выполняет. Они нужны, чтобы пояснить сложное место, оставить заметку или кратко объяснить смысл кода.

```
public class Car { no usages

    // Цвет машины
    String color; no usages

    /* Машина начинает движение */
    void drive() { no usages
        System.out.println("Машина едет");
    }

    /**
     * Останавливает машину
     */
    void stop() { no usages
        System.out.println("Машина остановилась");
    }
}
```

Здесь показаны три вида комментариев.

// - однострочный комментарий.

/* ... */ - многострочный.

/** ... */ - комментарий для документации.

Очень важная мысль - много комментариев в коде = плохой код. Хороший код должен объяснять сам себя!
Комментарии мы используем только **для крайне неочевидных сценариев** (разберем в будущем)

2.7 Чем исходный код отличается от результата выполнения программы?

Исходный код - это то, что написал программист. **Результат выполнения** - это то, что программа сделала после запуска.

```
class Car { no usages
    void drive() { no usages
        System.out.println("Машина едет");
    }
}
```

```
> Task :com.example.commander.config.Car.main()
Машина едет
```

```
BUILD SUCCESSFUL in 43s
```

То есть исходный код - это описание, инструкция, чертеж.

Результат выполнения - это уже действие.

Если держаться той же аналогии: исходный код - это **чертеж машины**, а результат выполнения - это момент, когда **машина уже поехала**.

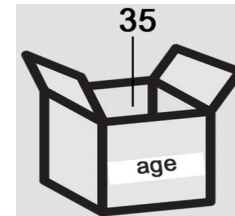


Запасайтесь терпением, мы взлетаем!

3.1 Что такое переменная?

Переменная - это **коробочка с именем**, в которой программа хранит данные.

```
int age = 35; 1 usage
boolean engineOn = true; 1 usage
char grade = 'A'; 1 usage
```



Например, коробочке age может лежать 35, в коробочке engineOn - true, а в коробочке grade - 'A'.

Главная мысль: **переменная нужна, чтобы программа могла что-то запомнить и потом с этим работать.**

3.2 Что значит объявить переменную?

Объявить переменную - значит **создать коробочку и сказать, что в ней можно хранить.**

```
int age; 1 usage
```

Здесь: int - тип данных age – имя переменной

То есть мы как будто подписали коробочку: **сюда можно класть только целые числа.**

3.3 Что значит присвоить переменной значение?

Присвоить значение - значит положить данные в переменную.

```
int age;
age = 20;
```

```
//можно в одну строку
int age = 20;
```

Объявление и присваивание - **не одно и то же.**

Объявление создает переменную, а **присваивание записывает** в нее значение.

3.4 Зачем переменной нужен тип данных?

Тип данных отвечает сразу на три вопроса:

1. **Что можно хранить** в переменной.
2. **Сколько памяти** под это нужно.
3. **Какие операции** с этим значением допустимы.

Если переменная имеет тип `int`, Java понимает, что внутри будет целое число, под него нужно 32 бита, и его можно складывать, вычитать, сравнивать. Если тип `boolean`, там могут быть только `true` или `false`.

Тип задает **правила хранения и работы с данными**.

3.5 Локальные переменные и поля класса

В Java переменные бывают не только “вообще”, а в разных местах программы. От этого зависит их поведение.

Вариант 1 - Поля класса

Если переменная объявлена **внутри класса, но вне методов**, это **поле**.

`model` и `maxSpeed` - поля.

Они принадлежат объекту класса `Car` и хранят его состояние.

```
public class Car { no usages
    String model; no usages
    int maxSpeed; no usages
}
```

Вариант 2 - Локальные переменные

Если переменная объявлена **внутри метода**, это **локальная переменная**.

```
public class Car { no usages
    //Поля
    String model; no usages
    int maxSpeed; no usages

    public void setSpeed() { no usages
        int maxSpeed = 10; //<--- Локальная переменная
    }
}
```

Главное различие!

Поле - это часть объекта или класса.

Локальная переменная - временная переменная внутри метода.

3.6 Какие примитивные типы данных есть в Java?

В Java есть **8 примитивных типов**. Это базовые типы языка, которые хранят **само значение**, а не ссылку на объект.

Напоминааааем, 1 байт = 8 бит

Тип	Размер	Что хранит	Диапазон / смысл
byte	8 бит	целое число	-128 .. 127
short	16 бит	целое число	-32768 .. 32767
int	32 бита	целое число	$-2^{31} .. 2^{31} - 1$
long	64 бита	целое число	$-2^{63} .. 2^{63} - 1$
float	32 бита	дробное число	одинарная точность
double	64 бита	дробное число	двойная точность
char	16 бит	один символ	0 .. 65535
boolean	не фиксирован спецификацией как обычный числовой размер	true / false	логическое значение

3.7 Чем отличаются целые числа, дробные числа, символы и логические значения?

Целые числа

Это числа **без дробной части**:

```
int age = 20;  
long population = 8000000L;
```

Дробные числа

Это числа **с дробной частью**:

```
double price = 99.9;  
float temperature = 36.6f;
```

Символ

Это **ровно один символ**

```
char grade = 'A';  
char digit = '7';
```

Логическое значение

Это только два варианта:

```
boolean isAdmin = true;  
boolean engineOn = false;
```


3.8 Значения переменных по умолчанию: где они есть, а где нет?

Поля класса и элементы массива получают значения по умолчанию

Если поле не инициализировать вручную, Java подставит стартовое значение сама. То же самое происходит с элементами массива

Пример и какие значения будут подставлены:

```
class User {  
    int age;  
    boolean active;  
    double salary;  
    char grade;  
    String name;  
}
```

Тип	Значение по умолчанию
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000'
boolean	false
ссылочный тип	null

Локальные переменные значения по умолчанию не получают!

У локальной переменной **нельзя** “надеяться, что Java что-то подставит”.

Если локальной переменной не присвоить значение до использования, будет ошибка компиляции.

```
public static void main(String[] args) {  
    int age;  
    System.out.println(age);  
}
```

Variable 'age' might not have been initialized

Initialize variable 'age' Alt+Shift+Enter More actions... Alt+Enter

int age

CalendarAgreagator.main

3.9 Что такое `boolean`, `char`, `int`, `double` и где они обычно используются?

`boolean` используют для состояний: да/нет, включено/выключено, найдено/не найдено.

```
boolean isPaid = true;
```

`char` используют, когда нужен один символ: буква, цифра, знак-разделитель.

```
char separator = ':';
```

`int` - основной тип для целых чисел. Это возраст, количество, индекс, счетчик.

```
int count = 15;
```

`double` - основной тип для дробных чисел. Его используют чаще, чем `float`, потому что у него выше точность

```
double weight = 72.5;  
double price = 199.99;
```

(но еще чаще и реальнее – `BigDecimal`, но об этом позже)

3.10 Что такое память и где вообще хранятся переменные?

Когда программа запускается, она работает не “в пустоте”, а в **оперативной памяти** компьютера, т.е. в **RAM**.

Именно туда во время работы попадают данные программы: числа, строки, объекты, массивы, временные переменные и служебная информация.

Если совсем просто, оперативную память можно представить как **очень большой набор ячеек**. У каждой ячейки есть адрес, и в этих ячейках можно что-то хранить

[ячейка][ячейка][ячейка][ячейка][ячейка][ячейка][ячейка]...

Важно не путать две вещи:

- **файл программы на диске** - это то, что просто лежит в памяти компьютера как файл;
- **данные программы в оперативной памяти** - это то, с чем программа работает прямо сейчас. При завершении программы этот кусок памяти очищается полностью



3.11 Как в памяти хранятся целые числа?

Чтобы понять, как в памяти хранятся byte, short, int и long, сначала нужно понять, как вообще число записывается в **двоичной системе**.

3.11.1 Десятичная и двоичная системы

В обычной жизни используется **десятичная система**:

число 583 означает: $5 \times 10^2 + 8 \times 10^1 + 3 \times 10^0$

То есть каждая позиция - это **степень числа 10**.

В памяти компьютера используется **двоичная система**.

Там каждая позиция - это **степень числа 2**.

Например, двоичное число: 1011

означает: $1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11$

то есть $1011_2 = 11_{10}$

3.11.2 Как переводить из двоичной системы в десятичную

Нужно умножить каждый бит на его степень двойки возведенный в степень позиции (начиная с нуля) и сложить.

$$11001 = 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 8 + 0 + 0 + 1 = 25$$

3.11.3 Как переводить из десятичной системы в двоичную

Нужно **делить число на 2** и записывать остатки.

Пример: переведем 13 в двоичную систему.

$$13 / 2 = 6 \text{ остаток } 1$$

$$6 / 2 = 3 \text{ остаток } 0$$

$3 / 2 = 1$ остаток 1

$$1 / 2 = 0 \text{ остаток } 1$$

Теперь читаем остатки **снизу вверх**: $13_{10} = 1101_2$

Проверка: $1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 8 + 4 + 0 + 1 = 13$

3.11.4 Что такое бит, байт и “32 ячейки”

Бит - это одна двоичная ячейка, в которой может лежать только 0 или 1.

Байт - это 8 бит.

Когда говорят, что `int` занимает **32 бита**, это значит, что под число выделяется **32 двоичные ячейки**.

int = 32 бита

[0][0][0][0][0][0][0][0][0][0][0][0][0][0][0]
[0][0][0][0][0][0][0][0][0][0][0][0][0][0][0]

3.11.5 Как хранится положительное целое число

Если число положительное, оно записывается в двоичном виде, а слева добавляется нулями до нужной длины.

Например, число **5 в двоичной системе**: 101

Если хранить его в **8 битах**: 00000101

Если хранить его в **32 битах** (int): 00000000 00000000 00000000 00000101

То есть положительное число - это просто двоичная запись числа, дополненная слева нулями. Такие нули называются **незначащими**.

3.11.6 Как хранится отрицательное число

Для byte, short, int, long Java использует формат **дополнительного кода** - **two's complement**.

Это значит, что отрицательные числа хранятся **не** как "минус и обычное число", а по специальному правилу.

Как **получить отрицательное число** в дополнительном коде:

1. берут двоичную запись числа x ;
2. меняют все биты наоборот: $0 \rightarrow 1, 1 \rightarrow 0$;
3. прибавляют 1.

Пример: как получить -5 в 8 битах

Сначала +5: 00000101

Меняем биты: 11111010

Прибавляем 1: 11111011

Значит: -5 = 11111011

Проверим: +5 = 00000101; -5 = 11111011

Именно так отрицательные числа и лежат в памяти.

3.11.7 Что такое знаковый бит

Если смотреть очень грубо, можно запомнить так:

- если старший бит 0, число обычно неотрицательное;
- если старший бит 1, число обычно отрицательное

Например, в 8 битах: 00000101 -> положительное число, 11111011 -> отрицательное число



3.11.8 Как определяется диапазон значений, которые мы можем хранить в переменных определенных типов?

Если под число выделено **n бит**, то всего можно собрать: 2^n разных комбинаций нулей и единиц.

Например, для `int`: $2^{32} = 4\,294\,967\,296$ разных вариантов значений

Общая формула всегда такая: от $-2^{(n-1)}$ до $2^{(n-1)}-1$, где **n** – количество бит, выделенные под тип

Примеры:

Для `int` (32 бита): от -2^{31} до $2^{31}-1$ (-2147483648 до 2147483647)

Для `byte` (8 бит): от -2^7 до 2^7-1 (от -128 до 127)

Почему в формуле справа -1, а слева нет

Разберем на примере `int`:

У `int` 32 бита.

Одна половина комбинаций **уходит на отрицательные числа**, другая - на неотрицательные. Но в неотрицательной части есть еще и 0.

Из-за этого получается: отрицательных значений - 2^{31} неотрицательных значений - тоже 2^{31} , но среди них уже есть 0.

Поэтому максимальное положительное число: $2^{31}-1$, а минимальное отрицательное: -2^{31}

3.11.9 А как в памяти хранится `char`?

`char` устроен иначе.

`char` - это **16-битный беззнаковый тип**, который хранит **код символа Unicode**.

То есть `char` - это не “маленькое целое число со знаком”, а число-код символа.

При этом в Unicode не бывает отрицательных кодов, значит и в `char` тоже

Например: `char c = 'A';`

Символ 'A' имеет код 65, значит в памяти хранится число 65.

Диапазон `char`: **0 .. $2^{16} - 1$ (0 .. 65535)**

Это может быть код буквы, цифры, пробела, знака и так далее.



3.12 Как в памяти хранятся float и double?

Не пугайтесь! Информация для ознакомления :)

Почему они так называются:

- float - от **floating point**, то есть “плавающая точка”;
- double - от **double precision**, то есть “двойная точность” (или же просто double float)

Из чего состоит такое число:

Любое float или double разбивается на три части: **знак** | **экспонента** | **мантисса**

Знак отвечает за плюс или минус, **экспонента** показывает, насколько нужно сдвинуть двоичную точку, **мантисса** хранит значащие биты числа.

Структура float - 1 бит знак | 8 бит экспонента | 23 бита мантисса

Структура double - 1 бит знак | 11 бит экспонента | 52 бита мантисса

Формула нормализованного числа:

Для понимания удобно пользоваться такой записью:

$$\text{value} = (-1)^{\text{sign}} \times 1.\text{mantissa} \times 2^{(\text{exponent} - \text{bias})}$$

Для float: bias = 127, для double bias = 1023

3.12.1 Что такое мантисса простыми словами?

Мантисса - это **основные значащие цифры** числа.

Например, число 5.75 в двоичной системе выглядит так: $5.75 = 101.11$

Нормализованная форма (по формуле): $101.11 = 1.0111 \times 2^2$

Теперь раскладываем: знак = 0, число положительное, экспонента = 2 (у нас 2 числа после запятой), для float храним не 2, а $2 + 127 = 129$, 129 в двоичном виде = 10000001, мантисса = биты после 1. → 0111...

То есть внутри float хранится не строка "5.75" и не “обычное число 5 и 75”, а вот такая конструкция:

```
sign = 0
exponent = 10000001
mantissa = 0111000000000000000000000000
```



3.13 Почему $0.1 + 0.2$ может дать 0.30000000000000004 ?

Потому что многие десятичные дроби **нельзя точно представить в двоичной системе**.

Точно так же, как $1/3$ нельзя точно записать конечной десятичной дробью, число 0.1 нельзя точно записать конечной двоичной дробью. Поэтому в памяти хранится ближайшее приближение.

```
System.out.println(0.1 + 0.2) → 0.30000000000000004
```

Это не “ошибка Java”, а особенность представления дробных чисел в памяти (почему – см. Раздел выше!)

Важный вывод:

`float` и `double` подходят для большинства вычислений с дробями, но **не подходят для точных денежных расчетов**.

3.14 Преобразование примитивных типов

Casting - это преобразование одного типа данных в другой.

Проще говоря, у нас есть значение одного типа, а мы хотим получить значение другого типа.

Например: из `int` сделать `double`; из `double` сделать `int`;

Важно сразу понять: **не любое преобразование безопасно, и не любое преобразование вообще возможно.**

Какие преобразования примитивных типов бывают?

У примитивов есть два основных варианта:

- **неявное преобразование** - Java делает его сама;
- **явное преобразование** - разработчик пишет его вручную.

Неявное преобразование

Оно происходит, когда значение маленького или "более узкого" типа помещается в более широкий тип без потери данных.

Здесь `int` автоматически превратился в `double`.

Почему это можно?

Потому что `double` умеет хранить больше вариантов значений, чем `int`.

```
int a = 10;  
double b = a;  
  
System.out.println(b); // 10.0
```

Явное преобразование

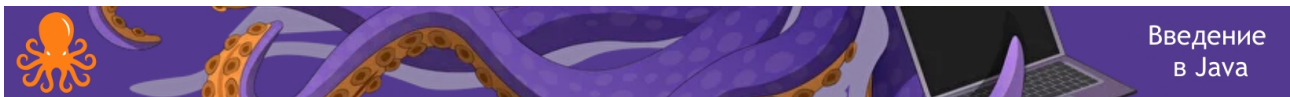
Нужно, когда мы хотим сделать наоборот: из более широкого типа перейти в более узкий.

Здесь Java сама такое не разрешит.

Нужно явно написать: **(int) a**

Это и есть явное преобразование, или cast.

```
double a = 9.8;  
int b = (int) a;  
  
System.out.println(b); // 9
```



Какие проблемы могут быть при сужающем преобразовании (явном)?

Может потеряться дробная часть

Здесь число не округлилось.

Оно просто **обрезалось**.

Это очень важный момент:

(int) 5.99 дает 5, а не 6.

```
double a = 5.99;
int b = (int) a;

System.out.println(b); // 5
```

Может потеряться часть числа вообще

На первый взгляд это выглядит странно. Почему не 130?

Потому что тип byte хранит значения только от -128 до 127.

Число 130 туда не помещается, и результат “переполняется”.

```
</> Java

int a = 130;
byte b = (byte) a;

System.out.println(b); // -126
```

Как правильно понимать безопасность преобразования примитивов?

Безопасное преобразование - это когда значение не теряется и не искажается.

Это безопасно

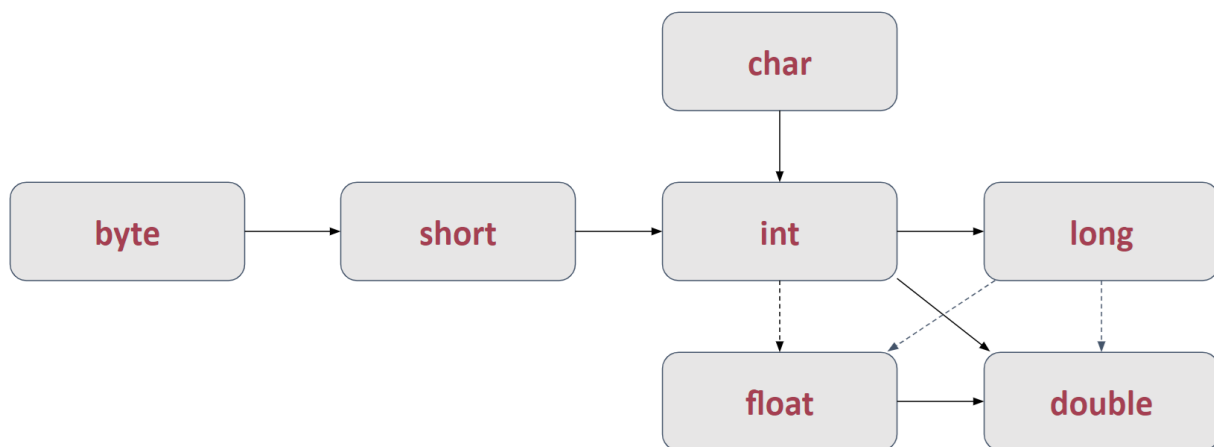
Это небезопасно (потерялась дробная часть)

```
int a = 100;
double b = a;
```

```
double a = 100.7;
int b = (int) a;
```

Практическое правило

Если идем: из меньшего типа в больший — обычно безопасно; из большего в меньший — нужно быть очень осторожным.



Схемы безопасного преобразования (из byte в short, из short в int и так далее). По сути всегда из меньшего типа – в больший.

Краткий вывод по разделу 3

Переменная - это коробочка с именем, в которой хранится значение.

Тип нужен, чтобы Java понимала, что можно хранить в переменной, сколько памяти выделять и какие операции разрешены.

Поля класса и элементы массива получают значения по умолчанию, а локальные переменные - нет.

Примитивные типы хранят само значение.

Целые типы хранятся в дополнительном коде, поэтому int имеет диапазон от -2^{31} до $2^{31} - 1$.

float и double состоят из знака, экспоненты и мантиссы, поэтому дробные числа часто хранятся с погрешностью.



4.1 Какие арифметические операторы есть в Java?

Основные арифметические операторы

Оператор	Что делает	Пример	Результат
+	сложение	5 + 3	8
-	вычитание	5 - 3	2
*	умножение	5 * 3	15
/	деление	10 / 2	5
%	остаток от деления	10 % 3	1

Пример:

```
int a = 10;  
int b = 3;  
  
System.out.println(a + b); // 13  
System.out.println(a - b); // 7  
System.out.println(a * b); // 30  
System.out.println(a / b); // 3  
System.out.println(a % b); // 1
```

Нюанс про деление: Если **обе части целые**, **результат** тоже **будет целым**.
Дробная часть просто отбросится.

Для дробного деления **левая или правая часть должны быть дробными**.

```
System.out.println(10 / 3); // 3  
System.out.println(10.0 / 3); // 3.3333333333333335
```

Оператор % часто сначала путает, но он очень полезен. Он не делит “по-настоящему”, а показывает, какой остаток у нас после деления.

```
System.out.println(10 % 3); // 1  
System.out.println(8 % 2); // 0
```

Например, так проверяют, **четное число** или нет:

```
int number = 8;  
System.out.println(number % 2 == 0); // true
```

Еще в Java есть **унарный минус** и **унарный плюс**. **Унарный** - значит применяется к одному числу. **“-”** **меняет знак числа**.

4.2 Чем отличается оператор присваивания = от оператора сравнения ==?

Оператор = - это **присваивание**. Он кладет значение в переменную.

```
int a = 5;  
System.out.println(a == 5); // true  
System.out.println(a == 3); // false
```

Оператор == - это **сравнение**. Он не записывает значение, а проверяет, равны ли две части выражения. Результат такого сравнения - true или false.

Главная разница очень простая:

- = - “запиши значение”;
- == - “сравни значения”.

Сравним:

В первой строке значение 10 записывается в x.

Во второй строке проверяется, равно ли x числу 10.

Ответ true, и он записывается в переменную b.

```
int x = 10;           // присваивание  
boolean b = x == 10; // сравнение
```

Нужно очень хорошо понять еще одну вещь: сравнение - это **логическое выражение**.

Логическое выражение - это выражение, результатом которого может быть только одно из двух значений:

- true — истина;
- false — ложь.

```
5 > 3      // true  
10 == 7    // false  
8 != 2     // true
```

То есть логическое выражение отвечает на вопрос: “да или нет?”, “верно или неверно?”.

Именно такие выражения потом используются в if, while, for и других конструкциях.

Здесь age >= 18 - это логическое выражение. Оно возвращает либо true, либо false.

```
int age = 20;  
  
if (age >= 18) {  
    System.out.println("Доступ разрешен");  
}
```



4.3 Какие операторы сравнения есть в Java и что они возвращают?

Операторы сравнения нужны, чтобы сравнивать значения. Результат их работы всегда один и тот же тип - boolean, то есть true или false.

Оператор	Что означает	Пример	Результат
==	равно	5 == 5	true
!=	не равно	5 != 3	true
>	больше	7 > 2	true
<	меньше	2 < 7	true
>=	больше или равно	5 >= 5	true
<=	меньше или равно	4 <= 6	true

Важно: операторы сравнения сами по себе ничего не выводят и ничего не меняют. Они только проверяют условие и возвращают true или false.

Например:

```
int x = 15;  
boolean result = x > 10;  
System.out.println(result); // true
```

```
int a = 10;  
int b = 7;
```

```
System.out.println(a == b); // false  
System.out.println(a != b); // true  
System.out.println(a > b); // true  
System.out.println(a < b); // false  
System.out.println(a >= b); // true  
System.out.println(a <= b); // false
```

Это удобно читать так:

“x > 10 - это правда?”

Если правда - получаем true. Если нет – false.



4.4 Как работают логические операторы &&, ||, !

Сначала важно понять основу.

Если выражение дает ответ “да” или “нет”, значит **это логическое выражение**.

Например:

Каждое из них возвращает либо true, либо false.

Логические операторы позволяют

соединять такие выражения между собой.

```
age >= 18
salary > 50000
name != null
```

Оператор && - логическое И

&& возвращает true только тогда, **когда оба** условия истинны.

Пример:

```
int age = 20;
boolean hasPassport = true;

System.out.println(age >= 18 && hasPassport); // true
```

Здесь **результат true**, потому что: $\text{age} \geq 18 \rightarrow \text{true}$; $\text{hasPassport} \rightarrow \text{true}$.

А **true && true** дает true.

Если хотя бы одно условие ложно, результат будет false.

```
System.out.println(20 >= 18 && false); // false
System.out.println(15 >= 18 && true); // false
```

Оператор || - логическое ИЛИ

|| возвращает true, если **хотя бы одно** условие истинно.

```
int age = 16;
boolean withParents = true;

System.out.println(age >= 18 || withParents); // true
```

Здесь первое условие ложное, второе истинное, но этого уже достаточно. $\text{false} || \text{true}$ дает true.



Оператор ! - логическое НЕ

! меняет значение на противоположное: true превращает в false, false превращает в true.

```
boolean isRain = true;  
System.out.println(!isRain); // false
```

Еще пример:

```
int age = 15;  
System.out.println(!(age >= 18)); // true
```

Сначала вычисляется `age >= 18`. Это false. Потом `!false` превращается в true.

4.4 Чем отличаются && и &, || и |?

&& и || - короткая проверка

Эти операторы сначала смотрят, нужно ли вообще проверять вторую часть.

Первая часть `a > 10` уже дает false.

А раз для **&&** нужно, чтобы **обе части** были **истинными**, Java понимает: дальше смысла проверять нет. Вторая часть даже не будет вычисляться.

```
int a = 5;  
  
if (a > 10 && a / 0 > 1) {  
    System.out.println("ok");  
}
```

То же самое с ||:

Первая часть уже **true**. Для **||** этого достаточно. Вторая часть не вычисляется.

```
int a = 5;  
  
if (a < 10 || a / 0 > 1) {  
    System.out.println("ok");  
}
```

& и | — без короткой проверки

Когда **&** и **|** применяются к boolean, они тоже могут работать как логические операторы, но **обе части выражения будут вычислены всегда**.

```
int a = 5;  
  
if (a > 10 & a / 0 > 1) {  
    System.out.println("ok");  
}
```

Здесь первая часть false, но Java все равно полезет вычислять вторую часть, и программа получит ошибку деления на ноль.



4.5 Как работают `i++` и `++i`? В чем между ними разница? Инкременты, декременты

Здесь появляется важное понятие — **инкремент**.

Инкремент — это увеличение значения переменной на 1.

Например, если было 5, после инкремента станет 6.

В Java для этого есть оператор `++`:

```
int i = 5;
++i;
System.out.println(i); // 6
```

Инкременты делятся на 2 вида:

Постфиксный (стоит после названия переменной): `i++` — сначала использовать, потом увеличить

Что произошло:

Сначала в `x` положили текущее значение `i`, то есть 5, потом `i` увеличили до 6.

```
int i = 5;
int x = i++;

System.out.println(i); // 6
System.out.println(x); // 5
```

Префиксный (стоит перед названием переменной): `++i` — сначала увеличить, потом использовать

Что произошло:

Сначала `i` увеличили до 6, потом уже это новое значение записали в `x`.

```
int i = 5;
int x = ++i;

System.out.println(i); // 6
System.out.println(x); // 6
```

Если очень просто:

- `i++` → “сначала старое значение, потом плюс один”;
- `++i` → “сначала плюс один, потом новое значение”.

```
int a = 10;

System.out.println(a++); // 10
System.out.println(a);   // 11
```

```
int b = 10;

System.out.println(++b); // 11
System.out.println(b);   // 11
```

Также есть **декремент** — уменьшение на 1. Для него используется `--`.

Правила полностью аналогичны инкременту.



4.6 Что такое составные операторы присваивания: +=, -=, *=, /=?

Составные операторы присваивания — это сокращенная запись, когда мы **берем старое значение переменной, что-то с ним делаем** и сразу **записываем результат** обратно в ту же переменную.

Например:

```
int a = 10;  
a = a + 5;
```

Эту запись можно сократить:

```
int a = 10;  
a += 5;
```

Оператор	Полная форма	Пример	Результат
+=	a = a + ...	a += 3	увеличить
-=	a = a - ...	a -= 3	уменьшить
*=	a = a * ...	a *= 3	умножить
/=	a = a / ...	a /= 3	разделить
%=	a = a % ...	a %= 3	остаток

Такие операторы делают код короче и удобнее для чтения. Особенно часто они встречаются в счетчиках, циклах, накоплении суммы и изменении значений.

```
int a = 20;  
  
a += 5;  
System.out.println(a); // 25  
  
a -= 3;  
System.out.println(a); // 22  
  
a *= 2;  
System.out.println(a); // 44  
  
a /= 4;  
System.out.println(a); // 11
```

5.1 Что такое условный оператор `if` и когда он используется?

if - это оператор, который позволяет программе принять решение: выполнять какой-то код или не выполнять.

Проще говоря, **if** проверяет условие. Если условие истинно, то есть дает `true`, код внутри выполнится. Если условие ложно, то есть дает `false`, код внутри не выполнится.

Пример:

Здесь условие - **age** `>= 18`.

Это логическое выражение. Оно возвращает либо `true`, либо `false`.

Если **age** равно 20, **условие истинно**, и строка "Доступ разрешен" выведется на экран.

Если бы было так:

то ничего бы не вывелось, потому что условие `15 >= 18` - **ложное**.

```
int age = 20;

if (age >= 18) {
    System.out.println("Доступ разрешен");
}
```

```
int age = 15;

if (age >= 18) {
    System.out.println("Доступ разрешен");
}
```

if используют везде, где программа должна вести себя по-разному в зависимости от ситуации:

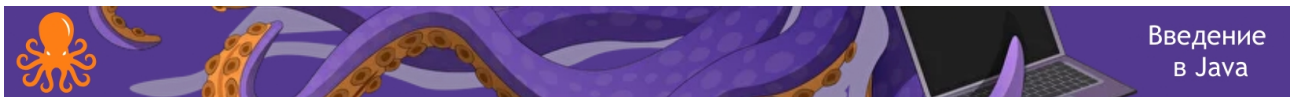
- если пользователь ввел правильный пароль - пустить;
- если на счете достаточно денег - провести списание;
- если число положительное - обработать одним способом;
- если товар есть на складе - оформить заказ.

Вместе с `if` мы можем использовать `else`. `else` означает: **"если условие не выполнилось, сделай вот это"**.

Здесь программа уже точно выберет один из двух вариантов.

```
int age = 15;

if (age >= 18) {
    System.out.println("Доступ разрешен");
} else {
    System.out.println("Доступ запрещен");
}
```



Также существует **else if**, когда нужно проверить несколько вариантов подряд.

Здесь Java идет сверху вниз и ищет первое подходящее условие. Как только подходящий вариант найден, **остальные проверки уже не выполняются**.

Главная идея if:

если условие верно - делаем одно, если нет- возможно делаем другое.

```
int score = 82;

if (score >= 90) {
    System.out.println("Отлично");
} else if (score >= 75) {
    System.out.println("Хорошо");
} else if (score >= 60) {
    System.out.println("Удовлетворительно");
} else {
    System.out.println("Не сдано");
}
```

5.2 Условный оператор Switch

switch удобен, когда мы сравниваем **одно значение** с несколькими конкретными вариантами.

Здесь проверяется одна переменная day, и для нее выбирается один из заранее известных вариантов.

switch лучше использовать, если:

- есть одна переменная;
- нужно сравнить ее с несколькими конкретными значениями;
- вариантов много, и хочется сделать код более читаемым.

```
int day = 3;

switch (day) {
    case 1:
        System.out.println("Понедельник");
        break;
    case 2:
        System.out.println("Вторник");
        break;
    case 3:
        System.out.println("Среда");
        break;
    default:
        System.out.println("Неизвестный день");
}
```

Например: коды команд, дни недели, роли пользователя, статусы заказа.

if-else - для **гибких и сложных** условий.

switch - для **выбора одного из нескольких точных** вариантов.

5.3 Циклы. Какие циклы есть в Java?

Цикл - это конструкция, которая позволяет выполнять один и тот же код несколько раз.

Циклы нужны очень часто. Например:

- вывести числа от 1 до 10;
- пройти по массиву;
- читать данные, пока они поступают;
- повторять действие, пока не выполнится условие.

В Java есть три основных цикла: **for**, **while**, **do while**

Все они делают одну общую вещь: повторяют код. Но устроены немного по-разному.

5.4 Чем отличаются for, while и do while?

for обычно используют, когда **заранее понятно, сколько раз** нужно повторить действие.

Пример:

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Результат:

```
1  
2  
3  
4  
5
```

Разберем по частям:

```
for (int i = 1; i <= 5; i++)
```

Здесь: `int i = 1` — начальное значение счетчика;

`i <= 5` — условие, пока оно истинно, цикл работает;

`i++` — что делать после каждой итерации, здесь увеличивать `i` на 1.

Итерация — это одно полное выполнение тела цикла.

То есть в примере выше: первая итерация: `i = 1`, вторая итерация: `i = 2`, и так далее..

while используют, когда заранее неизвестно, сколько раз придется повторять код, но известно условие, пока цикл должен работать. Выполняется он ровно до того момента, пока условие внутри скобок – true. При первой же проверке с false – цикл завершается.

Пример:

```
int i = 1;

while (i <= 5) {
    System.out.println(i);
    i++;
}
```

Результат будет тот же:

```
1
2
3
4
5
```

Здесь логика такая:

- Сначала проверяется условие $i \leq 5$;
- если оно true, тело цикла выполняется;
- потом снова идет проверка условия;
- если условие стало false, цикл останавливается.

while удобно использовать, когда **программа должна крутиться**, пока что-то не произойдет. Например: пока пользователь не введет правильное значение, пока не найден нужный элемент, пока идет чтение данных.

do while похож на while, но есть одна важная разница: он сначала выполняет код, а потом проверяет условие. Очень важно – при любых условиях, содержимое внутри do while **выполнится МИНИМУМ 1 раз**.

Пример:

```
int i = 1;

do {
    System.out.println(i);
    i++;
} while (i <= 5);
```

Результат снова будет:

```
1
2
3
4
5
```

5.5 Что делают break и continue?

Это специальные **операторы для управления циклом**.

break - полностью завершает цикл:

Пример:

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) {  
        break;  
    }  
    System.out.println(i);  
}
```

Результат:

1
2
3
4

Когда i стало равно 5, **сработал break**, и цикл **сразу остановился**.

break используют, когда **дальше крутить цикл уже не нужно**. Например, нашли нужный элемент - значит можно закончить поиск.

continue - не завершает цикл полностью.

Он просто пропускает текущую итерацию и переходит к следующей.

Пример:

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) {  
        continue;  
    }  
    System.out.println(i);  
}
```

Результат:

1
2
4
5

Когда $i == 3$, команда `continue` заставляет цикл пропустить вывод числа 3 и перейти дальше.

Простое различие:

`break` — “выходим из цикла совсем”.

`continue` — “эту итерацию пропускаем, но цикл продолжается”.